Student performance prediction

Project Objective: Transitioning from predicting continuous scores to categorical outcomes, this project pipeline builds a classification model to accurately predict a student's `final_grade` (e.g., A, B, C, D, E, F) based on their academic and behavioral features.

Tools & Technologies

Python & Pandas

Python served as the core logic engine, while Pandas was utilized to ingest the CSV file, handle missing data, and perform one-hot encoding for text categories.

Matplotlib & Seaborn

These graphical libraries generated visual insights, including class distributions and correlation heatmaps to visualize the relationships between numeric features.

Scikit-Learn

The industry-standard machine learning library used to split the dataset, build the Random Forest Classifier, and calculate evaluation metrics.

GridSearchCV

An algorithmic tool used to automatically test hundreds of different model settings (hyperparameters) to find the most accurate configuration.

Phase 1: Data Processing Pipeline

1. Feature Engineering & Cleaning

Identifying features like `student_id` as pure noise, they were removed from the dataset. Missing numerical values were filled using the median, while missing categorical values were replaced with the most frequent class (mode).

2. Target Isolation

To prevent "data leakage" (where the model cheats by looking at the answer), we completely removed the `overall_score` column before training. The model must learn to predict the `final_grade` using only the base metrics.

3. One-Hot Encoding

Machine learning models cannot read text. Using `pd.get_dummies`, all categorical text columns were transformed into binary (0 or 1) numerical columns, making them readable for the Random Forest algorithm.

Phase 2: Modeling & Evaluation

4. Stratified Splitting

The dataset was split into an 80% Training set and a 20% Testing set. We utilized *stratified splitting* to ensure that rare grades (like 'F') appeared in the same proportion in both sets, preventing the model from becoming biased toward common grades.

5. Random Forest Classification

We deployed a Random Forest Classifier—an algorithm that builds hundreds of mathematical "decision trees" and takes a majority vote to determine the final predicted grade. It is highly resistant to overfitting and handles complex relationships well.

6. Hyperparameter Tuning

We applied GridSearchCV to test different combinations of `n_estimators` (number of trees) and `max_depth` (how complex each tree can get). The model automatically selected the configuration that yielded the highest accuracy.

Understanding the Confusion Matrix

Because this is a multi-class problem (A through F), simple accuracy doesn't tell the whole story. The generated **Confusion Matrix** heatmap visualizes exactly where the model makes mistakes. For instance, it reveals if the model is perfectly predicting 'A's and 'F's, but occasionally confusing a borderline 'B' student for a 'C' student.

Final Model Performance

The classification model was evaluated strictly on unseen testing data. The key metric is **Accuracy** (the exact percentage of grades the model guessed correctly).

Evaluation Stage	Metric Type	Achieved Value
Baseline Model	Accuracy (Base Trees)	`0.7577`
Tuned Model	Accuracy (Optimized)	`0.7577`
Best Parameters	Max Depth / N-Estimators	`max_depth: None, n_estimators: 50`

Conclusion

By pivoting from regression to classification, this pipeline successfully categorizes students into distinct